Reproducible Cybersecurity Labs with Nix/NixOS

Nix fundamentals → Packaging → NixOS config → Custom ISO Builds

MEMSEC - DEFCON 901

@0x6a64

John Dorion

@0x6a64 - claybeard on discord

- Sales engineer, cybersecurity, cloud infrastructure & long time Linux nerd
- NixOS daily driver - my entire setup is on GitHub
- Frequent CTF competitor
- Not a Nix savant
- Pottery, cycling & roguelikes
- Fun fact!

github.com/0x6a64/

linkedin.com/in/johndorion/

BSides Memphis VP & Ops Lead

Why Nix is Cool & Reproducibility Matters

In infosec and IT operations, your environment IS your capability. Nix makes it reliable.

⚡ Setup Everything in Minutes*

Boot an identical lab on your laptop, a teammate's box, or a cloud VM in minutes. No runbook, no wiki, no drift

🔁 Perfectly Shareable

Point anyone at a git repo. They get the same tool versions, same config, same environment. **BIT FOR BIT** reproducibility 80-90% of the time

📜 Your Lab is a Git Repo

Every tool addition, config tweak, and version bump is a commit. Diff, review, and roll back your environment like code. Can include your home directory config files as well

🤖 “It just works”

Describe the end state you want and Nix figures out how to reach it. If it can’t it will produce (really ugly) errors

Supports multiple levels of rollbacks

What is Nix? (What isn't Nix???)

Package Manager

Install any package from nixpkgs, over 100,000 of them, without touching your existing system. Works alongside apt, brew, pacman, etc.

Functional Programming Language

Programs are built by composing pure functions - functions that always return the same output for the same input and produce no side effects.

Build system

Every package builds in a sandbox with explicit pinned dependencies. No implicit env vars, no assumptions. How it is defined in code is how it is built.

Key insight: Nix separates ***what*** you want from ***how*** to get it — and verifies the result cryptographically.

nixpkgs 100k+ pkgs	home-manager user config	nix-darwin macOS	NixOS full OS

The Nix Store: /nix/store

Every package lives at a unique, content-addressed, immutable path:

```
/nix/store/abcd1234...-openssl-3.0.2/bin/openssl
```

Content-Addressed

The hash derives from **ALL** inputs: source code, dependencies, build flags. Change anything and you get a completely different path.

Immutable

Once written, packages are never modified. The store is read-only after a build completes. Nothing can silently change under you.

Composable

Dependencies reference each other by full store path — no PATH tricks needed. Total isolation between different versions of the same library.

Derivations: The Building Block (Vimjoyer Video)

A compiled build recipe of a package. It knows what it's building, how it's building it, and what other derivations it depends on

Inputs

- Source tarball OR Cached binary
- Dependencies
- Build flags
- Patches
- Environment variables

→

Builder (sandbox)

- Isolated network
- No /home or /tmp
- Deterministic environment (Cleanroom)
- Set $out path

→

Output

- /nix/store/
  - {hash}-pkg/
    - bin/
    - lib/
    - share/

Nix Language Basics

nix

```nix
# Nix is a lazy, purely functional expression language

# Strings and interpolation
name    = "openssl";
version = "3.0.2";
url     = "https://example.com/${name}-${version}.tar.gz";

# Attribute sets  (like JSON objects / dicts)
myPkg = {
  pname   = "openssl";
  version = "3.0.2";
  license = "ASL20";
};

# Functions  — everything is a function  (arg: body)
greet = name: "Hello, ${name}!";
add   = a: b: a + b;

# let...in  (local scope)
let x = 1; y = 2; in x + y    # => 3

# with  (bring attrs into scope — used everywhere for packages)
with pkgs; [ curl wget jq nmap ]
```

Nix Flakes: Modern Reproducible Nix (⚠️ Unstable?)

Flakes are the standard way to write reproducible Nix projects today

📌 Pinned inputs

flake.lock pins every dependency to an exact git revision. Rebuilding weeks later gives identical results — no surprise updates.

📦 Explicit outputs

Packages, NixOS configs, devShells — all declared in one flake.nix outputs function. Nothing implicit, nothing hidden.

🔄 Composable

Import other flakes as inputs. Build on nixpkgs, home-manager, or your own overlays as composable building blocks.

🏗️ Standard UX

nix build .#myPkg | nix run .#tool | nix develop — consistent commands across every flake-based project.

flake.nix — Structure

nix

```nix
{
  description = "My NixOS config + security tooling";

  inputs = {
    nixpkgs.url          = "github:NixOS/nixpkgs/nixos-unstable"; #Can MIX unstable + versioned repo
    nixpkgs-stable.url   = "github:NixOS/nixpkgs/nixos-25.11";
    home-manager.url      = "github:nix-community/home-manager";
    sops-nix.url          = "github:Mic92/sops-nix";

    # All community flakes share OUR nixpkgs — one package set ☭
    home-manager.inputs.nixpkgs.follows = "nixpkgs";
    sops-nix.inputs.nixpkgs.follows     = "nixpkgs";
  };

  outputs = { nixpkgs, home-manager, sops-nix, stylix, disko, ... }: {

    nixosConfigurations.myHost = nixpkgs.lib.nixosSystem {
      system  = "x86_64-linux";
      modules = [
        ./hosts/myHost/configuration.nix
        sops-nix.nixosModules.sops         # secrets management
        home-manager.nixosModules.home-manager  # dotfiles & user pkgs
      ];
    };
  };
}
```

Nix Packaging — Writing a Derivation


```nix
{ lib, stdenv, fetchurl, openssl, cmake }:
stdenv.mkDerivation rec {
  pname   = "my-security-tool";
  version = "1.2.3";

  # Source pinned by hash — tamper-evident
  src = fetchurl {
    url    = "https://example.com/${pname}-${version}.tar.gz";
    sha256 = "sha256-abc123def456...";
  };

  nativeBuildInputs = [ cmake ];    # build-time deps
  buildInputs       = [ openssl ];  # run-time deps

  # Standard phases: unpack → patch → configure → build → install
  # stdenv handles these — only override what differs:
  installPhase = ''
    make install PREFIX=$out
  '';

  meta = with lib; {
    description = "A reproducible security tool";
    license     = licenses.mit;
    platforms   = platforms.linux;
  };
}
```

Nix Project Directory Structure (Vimjoyer Video)

A minimal flake-based NixOS project for ISO generation

```
nixiso/
├── flake.nix
├── flake.lock
├── .envrc
├── iso-configuration.nix
└── modules/
    ├── packages.nix
    ├── users.nix
    ├── gnome.nix
    ├── theming.nix
    └── dconf.nix
```

flake.nix

Entry point — declares inputs (nixpkgs) and outputs (ISO, system config, devShells).

flake.lock

Auto-generated, commit this. Pins every dependency to an exact git SHA.

.envrc (Optional)

direnv integration — auto-activates your Nix dev shell on cd.

iso-configuration.nix

Declares the ISO: tools, users, desktop, auto-login settings.

modules/

Logical split of config — easy to customize or disable individual sections.

Direnv + Automatic Shell Loading

Auto-activate your Nix dev shell the moment you cd into a project directory

```
# .envrc

use flake
```

```
# flake.nix outputs:
devShells.x86_64-linux.default =
  pkgs.mkShell {
    name = "security-lab";
    buildInputs = with pkgs; [
      nmap python3 pwntools
      burpsuite gobuster
    ];
    shellHook = ''
      echo "Lab shell loaded"
    '';
  };
```

Setup workflow:

1. **Install direnv, nix-direnv**
   Available in nixpkgs — add to your system packages
2. **Add .envrc**
   One line in your project root: use flake
3. **direnv allow**
   Authorize the .envrc the first time
4. **cd away, cd back**
   Shell deactivates on exit, activates on entry
5. **Zero manual nix develop**
   Tools appear automatically every time you enter the directory

Nix vs Containers and Virtual Machines

They solve different problems — and they work well together

Docker / Podman

Runtime isolation via Linux namespaces and cgroups. Packages an app + its dependencies into a portable image. Solves 'runs everywhere at runtime' — the image travels, not the build recipe.

Kubernetes

Container orchestration. Manages running containers across fleets of machines — scheduling, scaling, networking. Operates on Docker/OCI images. Not a build system.

VMs / Hypervisors

Full machine isolation via hardware virtualization. Heaviest weight. Great for strong isolation but expensive, slow to boot, and still suffers from snapshot rot/drift over time.

NixOS / Nix

- Reproducibility at build time, not just runtime — the build recipe is the artifact, not the image
- Declarative system config — OS, services, users, packages all in version control
- No container daemon needed — packages are isolated by the Nix store, not namespaces***
- Generations and rollback at the OS level — not just the app layer
- Nix can BUILD OCI/Docker images from a flake — best of both worlds

They complement each other: NixOS can run Docker/Podman containers, and Nix can build Docker images reproducibly with `pkgs.dockerTools.buildImage`

Nix Runs on Any Platform

Nix is not NixOS. Install the package manager on whatever you already run today

```
$ sh <(curl -L https://nixos.org/nix/install)  # that's it, that’s bad security ;)
```

Ubuntu / Debian

Any version, any arch

Fedora / RHEL

dnf and Nix coexist fine

Arch Linux

pacman + Nix side by side

macOS

Intel + Apple Silicon

WSL2 (Windows)

Ubuntu or any WSL distro

NixOS

Nix is the foundation of the OS

Community projects like NixOS-Anywhere help to deploy to all major cloud platforms or networked devices

NixOS

A Linux distribution where the entire OS is declared, not configured

Traditional Distros vs NixOS

Two different philosophies: Imperative or “ClickOps” vs. declarative state

	Traditional Linux Distro		NixOS
Packages	sudo apt install / dnf install — modifies shared mutable state imperatively (install things by hand)	→	Declare packages in configuration.nix — Nix builds and activates them atomically
Config	Scattered across /etc, managed by hand or Ansible — state drift happens silently	→	All config lives in version-controlled Nix files — drift is structurally prevented
Upgrades	In-place upgrade may silently break things — rollback is painful or impossible (although getting better w/ btrfs)	→	Every rebuild creates a new generation — boot the previous one from your bootloader anytime
Sharing	Causes "Works on my machine" problems and lengthy procedures to get machines in similar states	→	Share a git repo — anyone runs nix build and gets the identical system

NixOS: Declarative OS Configuration

Instead of running commands you describe the desired system state in configuration.nix

1

Modules

Every OS aspect is a module: networking, users, services, bootloader, packages. Each has typed options.

2

Declarative

`nixos-rebuild switch` reads your config and builds your current system to match it

3

Atomic

Each rebuild is a new generation. Boot the previous one from the bootloader at any time without data loss.

4

Testable

`nixos-rebuild test` applies config without touching the bootloader entry and `nixos-rebuild build-vm` launches config as a VM

Rebuild Cycle

1. Edit configuration.nix
2. nixos-rebuild switch
3. Nix evaluates + builds
4. New generation activated
5. Rollback anytime

Configuration.nix — Example

nix

```nix
{ config, pkgs, ... }: {

  # System packages available to every user
  environment.systemPackages = with pkgs; [
    nmap  metasploit  wireshark  burpsuite
    sqlmap  gobuster  hashcat  john
    git   tmux   neovim   ripgrep  bat
  ];

  # Services
  services.openssh = {
    enable   = true;
    settings.PasswordAuthentication = false;
  };

  # User with group memberships and SSH key
  users.users.hacker = {
    isNormalUser = true;
    extraGroups  = [ "wheel" "docker" "wireshark" ];
    openssh.authorizedKeys.keys = [ "ssh-ed25519 AAAA..." ]; # or use sops-nix
  };

  # Kernel tuning — declarative and version-controlled
  boot.kernel.sysctl."net.ipv4.ip_forward" = 1;
}
```

ISO Generation

Building bootable, reproducible cybersecurity lab images

Building a Custom NixOS ISO

Nix can build a bootable ISO from your flake config in a single command:

```
$ nix build .#nixosConfigurations.{live-iso-hostname}.config.system.build.isoImage
```

01

Write iso-configuration.nix

Declare your tools, users, desktop, and ISO-specific settings (installer profile, auto-login, squashfs compression).

02

Reference it in flake.nix

Add a nixosConfiguration entry using nixpkgs.lib.nixosSystem with the iso module imported. (for the installer)

03

Nix build

Nix fetches, builds, and assembles everything. Output lands at result/iso/nixos-*.iso — ready to flash or boot.

04

Flash, boot, or ship

dd to USB, boot in QEMU, or upload as a GitHub Release artifact for teammates to grab by URL.

Automation Opportunity: **Build your images using GitHub Actions to keep current images available**

iso-configuration.nix

nix

```nix
{ config, pkgs, modulesPath, ... }: {

  imports = [
    "${modulesPath}/installer/cd-dvd/installation-cd-graphical-gnome.nix"
  ];

  isoImage.isoName             = "nixos-security-lab.iso";
  isoImage.squashfsCompression = "zstd";

  # Full security toolkit — declared, not installed manually
  environment.systemPackages = with pkgs; [
    nmap  netcat-gnu  socat  masscan
    wireshark  tcpdump  tshark
    metasploit  sqlmap  gobuster  ffuf
    hashcat  john  hydra
    ghidra-bin  radare2  gdb  pwndbg
    python3  pwntools  impacket
    git  tmux  neovim  ripgrep
  ];

  # Auto-login for lab convenience
  services.xserver.displayManager.autoLogin = {
    enable = true;
    user   = "nixos";
  };
}
```

Testing with Nix Build (No Full ISO Needed)

bash

```bash
# Build a throwaway QEMU VM from your ISO config — no full ISO burn needed
# Perfect for iterating before committing to a real image
# Hostname refers to the hostname of the live-iso

# One-liner: build and launch in one shot
nix run .#nixosConfigurations.{hostname}.config.system.build.vm
nixos-rebuild build-vm --flake ~/flake-dir

# Or build first, run separately
nix build .#nixosConfigurations.{hostname}.config.system.build.vm
./result/bin/run-isoImage-vm

# Custom RAM / CPU count
QEMU_OPTS="-m 4G -smp 4" ./result/bin/run-{hostname or nixos}-vm

# Shared folder between host and VM
QEMU_OPTS="-virtfs local,path=/tmp/shared,mount_tag=host0,security_model=mapped" \
  ./result/bin/run-{hostname or nixos}-vm
# Inside VM: mount -t 9p -o trans=virtio host0 /mnt

# Happy with it? Swap build.vm → build.isoImage for the real thing
nix build .#nixosConfigurations.{hostname}.config.system.build.isoImage
# => ./result/iso/nixos-*.iso
```

Use Cases: Security Labs & CTFs

CTF Team Environment/Pentesting Lab

Pre-load a per-event ISO, config, or modules with exactly the tools you need. Share the URL. Everyone boots the same environment

Malware Analysis

Air-gapped immutable NixOS VM with Ghidra, cutter, strace. Nuke and rebuild between samples.

Workshops & Training

Send students an ISO link. Everyone boots the identical environment. Zero setup anxiety, zero lost workshop time.

Ephemeral Attack/Target Boxes

Create ISOs and configurations that intentionally use outdated versions or bad configurations. Or fully rig machines to come online as part of the challenge

Community Security NixOS Projects

Because Nix makes it easy, the community has built security-focused configs you can fork today

PwNixOS

`exploitoverload/PwNixOS`

Productivity-focused hacking NixOS flake with network analysis and pentesting tools.

Athena-Nix

Athena-OS/athena-nix

Security-focused NixOS distro/flake. Used to be based on Arch Linux

Tracking security tools in nixpkgs

PenNix

`linuxmobile/PenNix`

Modular pentesting container — hundreds of up-to-date tools for CTFs and red teaming.

Pwnix (Archived)

`Arian-D/pwnix`

Security-focused NixOS distro. Addresses pain points of traditional pentesting distros.

RedNix/RedNixOS(Abandoned?)

`redcode-labs/RedNix`
`redcode-labs/RedNixOS`

Hackable NixOS container and devShells collection. Compose into your own setup.

Full NixOS-based distro for cybersecurity. Focused on offensive security tooling.

maxos (Abandoned?)

`rascal999/maxos`

Pentest-focused NixOS config with an emphasis on operational usability and customization.

And more! Kai - linuxmobile/kai - a nix flake curated for bug bounty hunting (Active!)

AI is the Nix Documentation You Need

The official docs are fragmented and often incomplete. AI bridges the gap effectively.

🔍 Find option names

"What's the NixOS option to enable wireguard?"

AI finds it faster than grepping the options/ the search website and provides context

🔄 Convert shell commands

Paste a bash install script, ask AI to convert it into Nix. Works surprisingly well.

🐛 Debug eval errors

Nix errors are cryptic. Paste the error + your config, AI usually spots the problem immediately.

📐 Module system patterns

"How do I conditionally enable a service based on another option?" Some examples are hard to find in docs or in public configs.

📦 Packaging help

AI can pretty much fully generate the packages from inputs, reference, and traditional build instructions

🧪 Nix repl + AI together

Use nix repl for interactive exploration, AI to explain what you find. Combine them for fast learning.

Resources & Next Steps

Official and “Official” Docs

- nixos.org/learn – NixOS manual
- nixos.wiki – Unofficial out-of-date wiki
- zero-to-nix.com – modern starter guide
- nix.dev – practical Nix patterns
- Vimjoyer

Community

- discourse.nixos.org – forum
- #nix:nixos.org on Matrix
- discord.gg/RbvHtGa – Unofficial Discord
- github.com/NixOS/nixpkgs

Search Tools

- search.nixos.org/packages
- search.nixos.org/options
- home-manager-options.extranix.com

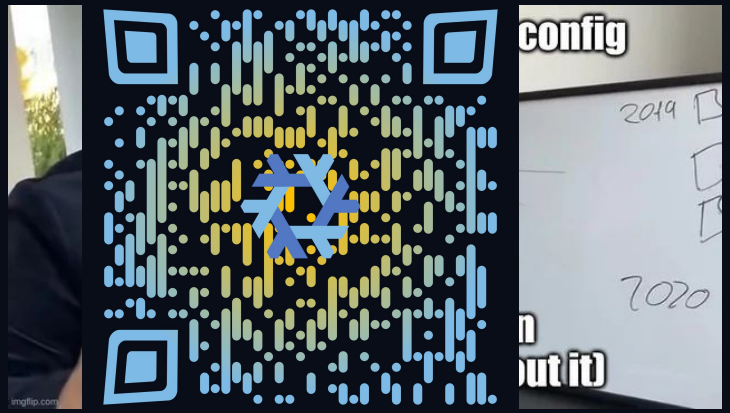
This Talk

- github.com/0x6a64/Nixiso
- github.com/0x6a64/NixOS-Public
- @0x6a64 on GitHub

$ _ Demo Time

What we can walk through:

1. Clone the Nixiso repo — explore the flake structure and modules
2. Walk through iso-configuration.nix: tools, desktop, users, auto-login
3. Show the GitHub Actions workflow — automated weekly builds (in theory)
4. Nix build & QEMU
5. Brief walkthrough of personal NixOS config (Cool Modules)

Questions?

Reproducible Cybersecurity Labs with NixOS